

/******

IR Java

Book for Beginners

Chapters List:

Chapter 1: Java Output & Display

Chapter 2: Variables & Data Types (Number, String, Date, Boolean)

Chapter 3: Operators & Numeric Expressions

Chapter 4: Math Library (Math.*)

Chapter 5: Numeric Types (Number & BigInt)

Chapter 6: Control Flow (if, switch, loops)

Chapter 7: String Processing

Chapter 8: DateTime & Temporal API

Chapter 9: Boolean Logic

Chapter 10: Functions & Execution Flow

Chapter 11: Objects & Structure Handling

Chapter 12: Map (Key-Value Collections)

Chapter 13: Arrays (Data Lists)

Chapter 14: Set (Unique Collections)

Chapter 15: Tips & Best Practices

Chapter 16: JSON & AJAX Data Handling

Chapter 17: Form CRUD Operations

Chapter 18: Modules (Import/Export System)

Chapter 19: Typed Arrays (Binary Data)

Chapter 20: Java Core APIs Cheat Sheet

* Chapter 0: JAVA BASICS (Equivalent Notes)

```
// Java is a high-level, object-oriented, compiled programming language.  
// It runs on JVM (Java Virtual Machine).
```

```
// 1. Java is Case Sensitive  
//    - firstName and FirstName are different variables
```

```
// 2. Java is Compiled Language  
//    - Code is compiled into bytecode before execution
```

```
// 3. Java is Event Driven (mainly in GUI / Web frameworks)
```

```
// 4. Java Core Data Types  
//    Primitive int, double, boolean, char, long, float, byte, short  
//    Non-Primitive String, Arrays, Objects, Collections
```

```
// 5. Java Comments  
//    // Single line  
//    /* Multi-line */
```

```
// 6. Java programs must be inside a class and main method
```

```
public class JavaCheatBook{  
  
    public static void main(String[] args) {  
  
        /*****  
        * CHAPTER 1: OUTPUT & DISPLAY  
        *****/  
  
        // ----- NUMBER -----  
  
        System.out.println(2);  
        // OUTPUT: 2  
  
        System.out.println(((Object)2).getClass().getSimpleName());  
        // OUTPUT: Integer (wrapper type concept)  
  
        Integer numObj = Integer.valueOf(231);  
        System.out.println(numObj);  
        // OUTPUT: 231  
  
        int num = Integer.parseInt("21");  
        System.out.println(num);  
        // OUTPUT: 21  
  
        System.out.println(6.05);  
        System.out.println(-8);  
        System.out.println(-5.009);  
  
        System.out.println(1 + " " + 9);  
        // OUTPUT: 1 9  
  
        System.out.println(11.03 + " " + (-4.91) + " " + 0.077);  
  
        System.out.println(5e20);    // scientific notation supported  
        System.out.println(5e25);
```

```

System.out.println(5.1e-6);

// BigInteger (Java equivalent of BigInt)
java.math.BigInteger big = new java.math.BigInteger("12345678901234567890");
System.out.println(big);
// OUTPUT: 12345678901234567890

// ----- STRING -----

System.out.println("A");
System.out.println('A');

String strObj = new String("Account");
System.out.println(strObj);
// OUTPUT: Account

System.out.println(String.valueOf(21));
// OUTPUT: 21

System.out.println("A cat".getClass().getSimpleName());
// OUTPUT: String

System.out.println("Text with\\nDouble\\nquotes");

System.out.println("Text with\\tSingle 'quotes");

System.out.println("Apple" + " Mango" + " Banana");

System.out.println("I am a \\\"Frontend Developer\\\"");

// ----- BOOLEAN -----

System.out.println(true);
System.out.println(false);

Boolean boolObj = Boolean.TRUE;
System.out.println(boolObj);

// ----- SYMBOL (NO DIRECT EQUIVALENT) -----
// Java does NOT have Symbol type
// ALTERNATIVE UUID

java.util.UUID id = java.util.UUID.randomUUID();
System.out.println("UUID: " + id);

// ----- BIGINT ALTERNATIVE -----
java.math.BigInteger big1 = new java.math.BigInteger("12345678901234567890");
java.math.BigInteger big2 = new java.math.BigInteger("12345678901234567890");

System.out.println(big1);
System.out.println(big2);

// ----- ARRAYS -----

int[] arr = {10, 20, 30};
System.out.println(java.util.Arrays.toString(arr));

// ----- NULL -----

String empty = null;

```

```

System.out.println(empty);

// ----- UNDEFINED (NO EQUIVALENT IN JAVA) -----
// Java variables must be initialized before use

// ----- DATE (JAVA) -----

java.util.Date now = new java.util.Date();
System.out.println(now);

java.time.LocalDate date = java.time.LocalDate.of(2026, 5, 8);
System.out.println(date);

java.time.LocalDate today = java.time.LocalDate.now();
System.out.println(today);

// ----- BOOLEAN OPERATIONS -----

System.out.println(10 > 5 && 5 < 8);
System.out.println(10 > 5 || 5 > 8);

System.out.println(5 == 5);
System.out.println(5 != 10);

System.out.println(10 < 20);
System.out.println(15 >= 15);

// ----- STRING CONCAT -----

int a = 3, b = 11;

System.out.println("Number#1 = " + a + " and Number#2 = " + b);

System.out.printf("Number#1 = %d and Number#2 = %d\n", a, b);

// ----- ERROR / WARN (JAVA STYLE) -----

System.err.println("Error message");
System.out.println("Warning message (no native warn type in Java)");

// ----- TABLE OUTPUT (ALTERNATIVE) -----

System.out.println("Name\tAge");
System.out.println("Ali\t25");
System.out.println("Ahmed\t30");

// ----- INPUT (PROMPT EQUIVALENT) -----

java.util.Scanner sc = new java.util.Scanner(System.in);
System.out.print("Enter name: ");
String name = sc.nextLine();
System.out.println("Hello " + name);

sc.close();

/*****
 * CHAPTER 2: VARIABLES & TYPES
 *****/

// LET equivalent -> local variables

```

```

int x;
int y = 10, z = 20;

System.out.println(x = 5);
System.out.println(y + z);

// CONSTANTS
final double PI = 3.1415;
final String fname = "John";

System.out.println(PI);
System.out.println(fname);

// DATE FORMATTING
java.time.LocalDate d = java.time.LocalDate.now();
String formatted = d.getDayOfMonth() + "-" + d.getMonthValue() + "-" + d.getYear();

System.out.println(formatted);

// BOOLEAN EXAMPLE
boolean isLoggedIn = true;
boolean isAdmin = false;

System.out.println("LoggedIn: " + isLoggedIn);
System.out.println("Admin: " + isAdmin);

boolean result = (10 < 20);
System.out.println("10 < 20 = " + result);
}
}

```

JAVA VERSION – CHAPTER 3, 4, 5 (OPERATORS & MATH)

```

import java.math.BigInteger;

public class Chapter3OperatorsMath{

    public static void main(String[] args) {

        /*****
         * CHAPTER 3: OPERATORS & EXPRESSIONS
         *****/

        // ----- SUM OF NUMBERS -----
        int number_1 = 5, number_2 = 10;
        int sum_numbers = number_1 + number_2;

        System.out.println("Sum of " + number_1 + ", " + number_2 + " is: " + sum_numbers);
        // OUTPUT: Sum of 5, 10 is: 15

        // ----- ADD & ASSIGN -----
        int num_51 = 51;

        num_51 = num_51 + 9;
        num_51 += 9;

        System.out.println("Final Value after additions = " + num_51);
    }
}

```

```
// OUTPUT: Final Value after additions = 69

// ----- STRING CONCAT NUMBERS -----
String strNumber_1 = "5", strNumber_2 = "10", strNumber_3 = "15";
String concat_numbers = strNumber_1 + strNumber_2 + strNumber_3

System.out.println("Concatenation of " + strNumber_1 + ", " + strNumber_2 +
    " and " + strNumber_3 + " is: " + concat_numbers);
// OUTPUT: 51015

// ----- DIFFERENCE -----
double number01 = 15.08, number02 = 10.11;
double difference_numbers = number01 - number02;

System.out.println("Difference of " + number01 + " and " + number02 +
    " is: " + difference_numbers);
// OUTPUT: 4.97

// STRING TO NUMBER SUBTRACTION (JAVA EXPLICIT CONVERSION)
String strNumber01 = "15.08";
double nNumber02 = 10.11;

double str_num_diff = Double.parseDouble(strNumber01) - nNumber02;

System.out.println("Difference is: " + str_num_diff);
// OUTPUT: 4.97

// ----- SUBTRACT & ASSIGN -----
int num_57 = 57, num_7 = 7;

num_57 = num_57 - num_7;
num_57 -= num_7;

System.out.println("Final Value after subtraction = " + num_57);
// OUTPUT: 43

// ----- MULTIPLICATION -----
double number$1 = 5.25, number$2 = 11;
double product_numbers = number$1 * number$2;

System.out.println("Product = " + product_numbers);
// OUTPUT: 57.75

// ----- MULTIPLY & ASSIGN -----
int num_10 = 10;

num_10 = num_10 * 5;
num_10 *= 5;

System.out.println("Final Value after multiplication = " + num_10);
// OUTPUT: 250

// ----- STRING MULTIPLY -----
```

```

double strNum01 = 5.8, strNum02 = 2.9;
double str_num_multiply= strNum01 * strNum02;

System.out.println("Multiplication= " + str_num_multiply);
// OUTPUT: 16.82


// ----- DIVISION -----
int number1 = 99, number2 = 11;
int division_numbers= number1 / number2;

System.out.println("Division= " + division_numbers);
// OUTPUT: 9


// STRING DIVISION (EXPLICIT)
int strNum1 = Integer.parseInt("20");
int strNum2 = Integer.parseInt("5");

int str_num_division= strNum1 / strNum2;

System.out.println("Division= " + str_num_division);
// OUTPUT: 4


// ----- DIVIDE & ASSIGN -----
int num_99 = 99, num_9 = 9;

num_99 = num_99 / num_9;
num_99 /= num_9;

System.out.println("Final Division Value = " + num_99);
// OUTPUT: 1


// ----- REMAINDER -----
int r1 = 99, r2 = 11;
int remainder = r1 % r2;

System.out.println("Remainder= " + remainder);
// OUTPUT: 0


// ----- POWER -----
int base = 2;
double power = Math.pow(base, 16);

System.out.println("Power = " + power);
// OUTPUT: 65536


// ----- INCREMENT / DECREMENT -----
int N = 9;
System.out.println(N++); // 9
System.out.println(N);   // 10

int X = 9;
System.out.println(++X); // 10

```

```

// ----- BITWISE -----
System.out.println("Bitwise AND: " + (15 & 5)); // 5
System.out.println("Bitwise OR: " + (15 | 3)); // 15

System.out.println("Even/Odd check: " + (12 & 1)); // 0
System.out.println("Even/Odd check: " + (7 & 1)); // 1

// ----- SHIFT OPERATORS -----
System.out.println(5 << 1); // 10
System.out.println(3 << 2); // 12

System.out.println(20 >> 1); // 10
System.out.println(20 >> 2); // 5

/*****
 * CHAPTER 4: MATH LIBRARY
 *****/

System.out.println(Math.PI);
System.out.println(Math.E);

System.out.println(Math.sqrt(25)); // 5
System.out.println(Math.cbrt(27)); // 3

System.out.println(Math.pow(3, 3)); // 27

// CONSTANTS
System.out.println(Math.LN2);
System.out.println(Math.LN10);
System.out.println(Math.log10(Math.E));

// AREA OF CIRCLE
double r = 7;
double area = Math.PI * r * r;
System.out.println("Circle Area = " + area);

// SIMPLE INTEREST
double P = 1000, R = 5, T = 2;
double SI = (P * R * T) / 100;
System.out.println("Simple Interest = " + SI);

// TRIGONOMETRY
double angle = Math.toRadians(30);
System.out.println("Sin = " + Math.sin(angle));
System.out.println("Cos = " + Math.cos(angle));

// TEMPERATURE CONVERSION
double celsius = 25;
double fahrenheit = (celsius * 9 / 5) + 32;
System.out.println("Fahrenheit = " + fahrenheit);

/****

```



```
double earth = 5.972e24;
double jupiter = 1.898e27;

System.out.println("Earth = " + earth);
System.out.println("Jupiter = " + jupiter);

// BIGINTEGER (EXACT VALUES)
BigInteger earthBig = new BigInteger("5972000000000000000000000");
BigInteger jupiterBig = new BigInteger("189800000000000000000000000");

System.out.println("Earth Big = " + earthBig);
System.out.println("Jupiter Big = " + jupiterBig);

// BIGINTEGERDIFFERENCE
System.out.println("Difference = " + jupiterBig.subtract(earthBig));

// BEST PRACTICE NOTE:
// Java uses double for scientific numbers (approx)
// BigInteger for exact integer precision
}
```

```

public class Chapter6ControlFlow{

    public static void main(String[] args) {

        /*****
        * 1. SPEED CHECKER
        *****/
        int speed = 120;

        if (speed > 100) {
            System.out.println("Overspeeding");
        } else if (speed >= 60) {
            System.out.println("Normal Speed ");
        } else {
            System.out.println("Too Slow ");
        }
        // OUTPUT: Overspeeding

        /*****
        * 2. GRADE CALCULATOR
        *****/
        int marks = 85;

        if (marks >= 90) {
            System.out.println("A");
        } else if (marks >= 80) {
            System.out.println("B");
        } else if (marks >= 70) {

```

```

        System.out.println("C");
    } else if (marks >= 60) {
        System.out.println("D");
    } else {
        System.out.println("Fail");
    }
}
// OUTPUT: B

/*****
 * 3. CIRCLE CALCULATOR
 *****/

double PI = 3.14159;
double radius = 5;
String type = "area";

if (type.equals("area")) {
    System.out.println(PI * radius * radius);
} else if (type.equals("circumference")) {
    System.out.println(2 * PI * radius);
} else {
    System.out.println("Invalid type");
}
// OUTPUT: 78.53975

/*****
 * 4. RECTANGLE CALCULATOR
 *****/

int length = 10, width = 5;
type = "area";

if (type.equals("area")) {
    System.out.println(length * width);
} else if (type.equals("perimeter")) {
    System.out.println(2 * (length + width));
} else {
    System.out.println("Invalid type");
}
// OUTPUT: 50

/*****
 * 5. TRIANGLE TYPE CHECKER
 *****/

int a = 10, b = 10, c = 10;

if (a == b && b == c) {
    System.out.println("Equilateral");
} else if (a == b || b == c || a == c) {
    System.out.println("Isosceles");
} else {
    System.out.println("Scalene");
}
// OUTPUT: Equilateral

/*****
 * 6. BILL CALCULATOR
 *****/

```

```

double amount = 1200;

if (amount >= 1000) {
    System.out.println(amount * 0.8);
} else if (amount >= 500) {
    System.out.println(amount * 0.9);
} else {
    System.out.println(amount);
}
// OUTPUT: 960.0

/*****
 * 7. LOGIN SYSTEM
 *****/

String user = "admin";
String pass = "1234";

if (user.equals("admin") && pass.equals("1234")) {
    System.out.println("Admin Login");
} else if (user.equals("user") && pass.equals("1111")) {
    System.out.println("User Login");
} else {
    System.out.println("Invalid Credentials");
}
// OUTPUT: Admin Login

/*****
 * 8. WEATHER ADVICE
 *****/

int temp = 40;

if (temp > 35) {
    System.out.println("Very Hot & p");
} else if (temp >= 20) {
    System.out.println("Pleasant p");
} else {
    System.out.println("Cold 'Dp'");
}
// OUTPUT: Very Hot & p

/*****
 * 9. MULTIPLICATIONTABLE (FOR LOOP)
 *****/

int tableNumber = 17;

for (int i = 1; i <= 10; i++) {
    System.out.println(tableNumber + " x " + i + " = " + (tableNumber * i));
}

/*****
 * 10. FORMULA LOOP (x2 + y/3)
 *****/

for (int x = 1, y = 3; x <= 5; x++, y += 3) {

    double z = (x * x) + (y / 3.0);

```

```

    System.out.println("x=" + x + " y=" + y + " z=" + z);
}

/*****
 * 11. FIND MAX IN ARRAY
 *****/
int[] arr = {10, 50, 30, 20};

int max = arr[0];

for (int i = 1; i < arr.length; i++) {
    if (arr[i] > max) {
        max = arr[i];
    }
}

System.out.println("Max: " + max);
// OUTPUT: 50

/*****
 * 12. WHILE LOOP TABLE
 *****/
int num = 5;
int i = 1;

while (i <= 10) {
    System.out.println(num * i);
    i++;
}

/*****
 * 13. VOTING ELIGIBILITY
 *****/
int age = 18;

if (age >= 18) {
    System.out.println("Eligible to Vote");
} else {
    System.out.println("Not Eligible");
}

/*****
 * 14. LARGEST OF THREE NUMBERS
 *****/
int a1 = 10, b1 = 25, c1 = 15;

if (a1 > b1 && a1 > c1) {
    System.out.println("A is largest");
} else if (b1 > a1 && b1 > c1) {
    System.out.println("B is largest");
} else {
    System.out.println("C is largest");
}

/*****

```

```

* 15. PRINT 1 TO N
*****/
int n = 5;

for (i = 1; i <= n; i++) {
    System.out.println(i);
}

/*****
* 16. ODD NUMBERS
*****/
for (i = 1; i <= 20; i++) {
    if (i % 2 != 0) {
        System.out.print(i + " ");
    }
}
System.out.println();

/*****
* 17. REVERSE NUMBER
*****/
int numRev = 123;
int rev = 0;

while (numRev > 0) {
    int digit = numRev % 10;
    rev = rev * 10 + digit;
    numRev = numRev / 10;
}

System.out.println("Reversed " + rev);
// OUTPUT: 321

/*****
* 18. COUNT DIGITS
*****/
int numCount = 12345;
int count = 0;

while (numCount > 0) {
    numCount = numCount / 10;
    count++;
}

System.out.println("Digits: " + count);
// OUTPUT: 5

/*****
* 19. POWER USING LOOP
*****/
int base = 4;
int result = 1;

for (i = 1; i <= 5; i++) {
    result *= base;
}

```

```
System.out.println("Result: " + result);
// OUTPUT: 1024
```

```

/*****
 * 20. EVEN / ODD INDEX IN STRING
 *****/
String str = "HELLO";

for (i = 0; i < str.length(); i++) {
    if (i % 2 == 0) {
        System.out.println(str.charAt(i) + " -> Even Index");
    } else {
        System.out.println(str.charAt(i) + " -> Odd Index");
    }
}

```

```

/*****
 * 21. ASCII EVEN / ODD
 *****/
String str2 = "ABC";

for (i = 0; i < str2.length(); i++) {
    int code = str2.charAt(i);

    if (code % 2 == 0) {
        System.out.println(str2.charAt(i) + " EVEN ASCII");
    } else {
        System.out.println(str2.charAt(i) + " ODD ASCII");
    }
}

```

```

/*****
 * NOTE (JAVA DIFFERENCE)
 *****/
// - Java does NOT support truthy/falsy checks
// - Must use explicit conditions (==, !=, >, <)
// - No automatic type conversion like JavaScript
}

}

/***** Chapter 8: DateTime Handling in Java (java.time API) *****/

```

```

/*****
CHAPTER 8: DATE & TIME OPERATIONS (JAVA VERSION)
Package Used: java.time (Modern Java Date API)

NOTE:
Java does NOT have Temporal API like JavaScript
Instead, we use:
- LocalDate
- LocalDateTime
- Period
- Duration
- ChronoUnit

```

```

*****/

import java.time.*;
import java.time.format.DateTimeFormatter;
import java.time.temporal.ChronoUnit;

public class DateTimeChapter8{

    public static void main(String[] args) {

        /*****
        1. CURRENT DATE & TIME
        *****/
        LocalDateTime now = LocalDateTime.now();
        System.out.println("Current DateTime: " + now);

        LocalDate today = LocalDate.now();
        System.out.println("Current Date: " + today);

        /*****
        2. DATE PARTS (YEAR / MONTH / DAY)
        *****/
        System.out.println("Year: " + today.getYear());
        System.out.println("Month: " + today.getMonthValue());
        System.out.println("Day: " + today.getDayOfMonth());

        /*****
        3. ADD DAYS
        *****/
        LocalDate plus10Days = today.plusDays(10);
        System.out.println(" +10 Days: " + plus10Days);

        /*****
        4. ADD MONTHS
        *****/
        LocalDate plus2Months = today.plusMonths(2);
        System.out.println(" +2 Months: " + plus2Months);

        /*****
        5. ADD YEARS
        *****/
        LocalDate plus1Year = today.plusYears(1);
        System.out.println(" +1 Year: " + plus1Year);

        /*****
        6. CUSTOM DATE CREATION
        *****/
        LocalDate custom = LocalDate.of(2026, 4, 24);
        System.out.println("Custom Date: " + custom);

        /*****
        7. FORMAT DATE -> "Mon, 5 April 2026"
        *****/
        DateTimeFormatter fmt =

```

```

        DateTimeFormatter ofPattern("EEE, d MMMM yyyy");

System.out.println("Formatted " + today.format(fmt));

/*****
8. YYYY-MM-DD FORMAT
*****/
DateTimeFormatter iso = DateTimeFormatter ofPattern("yyyy-MM-dd");
System.out.println("ISO Format: " + today.format(iso));

/*****
9. DATE DIFFERENCE
*****/
LocalDate d1 = LocalDate of(2026, 4, 22);
LocalDate d2 = LocalDate of(2026, 6, 10);

long diffDays = ChronoUnit.DAYS.between(d1, d2);
System.out.println("Difference in Days: " + diffDays);

/*****
10. ADD / SUBTRACT DAYS
*****/
System.out.println("Add 1 Month: " + d1.plusMonths(1));
System.out.println("Subtract 10 Days: " + d1.minusDays(10));

/*****
11. MODIFY DATE
*****/
LocalDate modified = d1.withDayOfMonth(1);
System.out.println("Modified Date: " + modified);

/*****
12. COMPARISON
*****/
System.out.println("d1 > d2 ? " + d1.isAfter(d2));
System.out.println("d1 < d2 ? " + d1.isBefore(d2));
System.out.println("Equal ? " + d1.equals(LocalDate of(2026, 4, 22)));

/*****
13. NEXT 10 DAYS
*****/
System.out.println("==== NEXT 10 DAYS =====");
for (int i = 0; i < 10; i++) {
    System.out.println(today.plusDays(i));
}

/*****
14. EVEN / ODD DAY CHECK
*****/
System.out.println("==== EVEN / ODD DAY =====");
for (int i = 0; i < 7; i++) {
    LocalDate d = today.plusDays(i);
    int day = d.getDayOfMonth();

```



```

    if (day % 2 == 0)
        System.out.println(d + " -> Even Day");
    else
        System.out.println(d + " -> Odd Day");
}

/*****
15. PAST / FUTURE / TODAY
*****/
System.out.println("==== PAST / FUTURE ====");

for (int i = -3; i <= 3; i++) {
    LocalDate d = today.plusDays(i);

    if (d.isBefore(today))
        System.out.println(d + " -> Past");
    else if (d.isAfter(today))
        System.out.println(d + " -> Future");
    else
        System.out.println(d + " -> Today");
}

/*****
16. WEEKDAY / WEEKEND
*****/
System.out.println("==== WEEKDAY / WEEKEND ====");

for (int i = 0; i < 7; i++) {
    LocalDate d = today.plusDays(i);
    DayOfWeek day = d.getDayOfWeek();

    if (day == DayOfWeek.SATURDAY || day == DayOfWeek.SUNDAY)
        System.out.println(d + " -> Weekend");
    else
        System.out.println(d + " -> Weekday");
}

/*****
17. HOURS OF DAY
*****/
System.out.println("==== HOURS OF DAY ====");

for (int i = 0; i < 24; i++) {
    if (i < 12)
        System.out.println(i + ":00 AM");
    else
        System.out.println(i + ":00 PM");
}

/*****
18. LEAP YEAR CHECK
*****/
System.out.println("==== LEAP YEARS ====");

for (int y = 2020; y <= 2030; y++) {

```

```

        boolean isLeap = Year.isLeap(y);

        System.out.println(y + " -> " + (isLeap ? "Leap Year" : "Normal Year"));
    }

    /*****
    19. WORKING DAYS COUNT
    *****/
    LocalDate start = LocalDate.of(2026, 5, 1);
    LocalDate end = LocalDate.of(2026, 5, 10);

    int workingDays = 0;

    for (LocalDate d = start; !d.isAfter(end); d = d.plusDays(1)) {
        if (!(d.getDayOfWeek() == DayOfWeek.SATURDAY ||
            d.getDayOfWeek() == DayOfWeek.SUNDAY)) {
            workingDays++;
        }
    }

    System.out.println("Working Days: " + workingDays);

    /*****
    20. AGE CALCULATION
    *****/
    LocalDate dob = LocalDate.of(2000, 5, 10);

    Period age = Period.between(dob, today);

    System.out.println(
        "Age: " +
        age.getYears() + " Years " +
        age.getMonths() + " Months " +
        age.getDays() + " Days"
    );

    /*****
    21. QUARTER DETECTION
    *****/
    int month = today.getMonthValue();

    String quarter =
        (month <= 3) ? "Q1" :
        (month <= 6) ? "Q2" :
        (month <= 9) ? "Q3" : "Q4";

    System.out.println("Quarter: " + quarter);

    /*****
    22. EXPIRY DATE (3 MONTHS)
    *****/
    LocalDate expiry = today.plusMonths(3);
    System.out.println("Expiry Date: " + expiry);

    /*****/

```

23. BILLING CYCLE

```
*****/  
LocalDate billing = LocalDate.of(2026, 1, 1).plusMonths(6);  
System.out.println("Billing End Date: " + billing);
```

```
/******
```

24. DATE COMPARISON CHECK

```
*****/  
LocalDate input = LocalDate.of(2026, 5, 5);
```

```
if (input.isEqual(today))  
    System.out.println("Today");  
else if (input.isBefore(today))  
    System.out.println("Past");  
else  
    System.out.println("Future");
```

```
/******
```

25. LAST DAY OF MONTH

```
*****/  
LocalDate lastDay = today.withDayOfMonth(today.lengthOfMonth());
```

```
System.out.println("Last Day: " + lastDay);
```

```
/******
```

26. DATE FORMAT (DD-MM-YYYY)

```
*****/  
DateTimeFormatter f = DateTimeFormatter.ofPattern("dd-MM-yyyy");
```

```
System.out.println("Formatted " + today.format(f));
```

```
/******
```

27. NEXT FRIDAY

```
*****/  
LocalDate nextFriday = today;
```

```
while (!nextFriday.getDayOfWeek().equals(DayOfWeek.FRIDAY)) {  
    nextFriday = nextFriday.plusDays(1);  
}  
System.out.println("Next Friday: " + nextFriday);
```

```
}  
}
```

```
/******
```

```
/******
```

CHAPTER 9 + 10 + 11 (JAVA VERSION)

BOOLEAN LOGIC + FUNCTIONS + OBJECT STRUCTURES

NOTES:

- Java does NOT support JavaScript-style dynamic functions (function expressions, arrow functions, new Function).
Instead we use:
 - ' static methods
 - ' class methods
 - ' interfaces/lambda (Java 8+)
- Java does NOT have JS objects ! we use POJO classes + Maps

```

import java.util.*;
import java.time.*;

/***** CHAPTER 9: BOOLEAN LOGIC *****/
class BooleanLogicChapter{

    // -----
    // TRAFFIC LIGHT (BOOLEAN FLAGS)
    // -----
    static void showLight(boolean red, boolean yellow, boolean green) {
        if (red) {
            System.out.println("STOP (Red Light ON)");
        } else if (yellow) {
            System.out.println("READY (Yellow Light ON)");
        } else if (green) {
            System.out.println("GO (Green Light ON)");
        } else {
            System.out.println("NO LIGHT ACTIVE");
        }
    }

    // CLEAN VERSION (STRING BASED - RECOMMENDED IN JAVA)
    static void showTrafficLight(String light) {
        switch (light.toLowerCase()) {
            case "red":
                System.out.println("STOP");
                break;
            case "yellow":
                System.out.println("READY");
                break;
            case "green":
                System.out.println("GO");
                break;
            default:
                System.out.println("INVALID LIGHT");
        }
    }

    // BOOLEAN ARRAY VERSION
    static void showLight(boolean[] lights) {
        if (lights[0]) System.out.println("STOP (Red Light)");
        else if (lights[1]) System.out.println("READY (Yellow Light)");
        else if (lights[2]) System.out.println("GO (Green Light)");
    }

    static boolean[] nextLight(boolean[] lights) {
        if (lights[0]) return new boolean[]{false, true, false};
        if (lights[1]) return new boolean[]{false, false, true};
        return new boolean[]{true, false, false};
    }
}

/***** CHAPTER 10: FUNCTIONS (JAVA METHODS) *****/
class FunctionChapter{

    // MultiplicationTable
    static void generateTable(int number) {

```

```

        for (int i = 1; i <= 10; i++) {
            System.out.println(number + " x " + i + " = " + (number * i));
        }
    }

    // WEATHER ADVICE
    static String weatherAdvice(int temp) {
        if (temp > 35) return "Very Hot";
        else if (temp >= 20) return "Pleasant";
        else return "Cold";
    }

    // CIRCLE CALCULATOR
    static double circleCalculator(double radius, String type) {
        double PI = 3.14159;

        if (type.equals("area")) {
            return PI * radius * radius;
        } else if (type.equals("circumference")) {
            return 2 * PI * radius;
        }
        return -1;
    }

    // RECTANGLE CALCULATOR
    static int rectangleCalculator(int length, int width, String type) {
        if (type.equals("area")) return length * width;
        if (type.equals("perimeter")) return 2 * (length + width);
        return -1;
    }

    // TRIANGLE TYPE
    static String triangleType(int a, int b, int c) {
        if (a == b && b == c) return "Equilateral";
        if (a == b || b == c || a == c) return "Isosceles";
        return "Scalene";
    }

    // BILL CALCULATOR
    static double calculateBill(double amount) {
        if (amount >= 1000) return amount * 0.8;
        if (amount >= 500) return amount * 0.9;
        return amount;
    }

    // SPEED CHECK
    static String checkSpeed(int speed) {
        if (speed > 100) return "Overspeeding";
        if (speed >= 60) return "Normal Speed";
        return "Too Slow";
    }

    // GRADE CALCULATOR (RANGE STYLE)
    static String getGrade(int marks) {
        if (marks >= 90 && marks <= 100) return "A";
        if (marks >= 80) return "B";
        if (marks >= 70) return "C";
        if (marks >= 60) return "D";
        if (marks >= 0) return "Fail";
        return "Invalid Marks";
    }

```

```

    }

    // LOGIN SYSTEM
    static String login(String user, String pass) {
        if (user.equals("admin") && pass.equals("1234")) return "Admin Login";
        if (user.equals("user") && pass.equals("1111")) return "User Login";
        return "Invalid Credentials";
    }
}

/***** CHAPTER 11: OBJECTS (JAVA CLASSES) *****/

// -----
// USER MODEL (POJO)
// -----
class User {
    String name;
    int age;
    String city;

    User(String name, int age, String city) {
        this.name = name;
        this.age = age;
        this.city = city;
    }
}

// -----
// PRODUCT MODEL
// -----
class Product {
    String name;
    int price;
    String brand;

    Product(String name, int price, String brand) {
        this.name = name;
        this.price = price;
        this.brand = brand;
    }
}

// -----
// MAIN DEMO CLASS
// -----
public class Main {

    public static void main(String[] args) {

        /***** BOOLEAN LOGIC *****/
        BooleanLogicChaptershowLight(true, false, false);
        BooleanLogicChaptershowTrafficLight("red");

        boolean[] lights = {true, false, false};
        BooleanLogicChaptershowLight(lights);

        lights = BooleanLogicChaptemextLight(lights);
        BooleanLogicChaptershowLight(lights);
    }
}

```

```

/***** FUNCTIONS *****/
FunctionChaptergenerateTable(5);

System.out.println(FunctionChapterweatherAdvice(40));
System.out.println(FunctionChaptercircleCalculator(5, "area"));
System.out.println(FunctionChapterrectangleCalculator(10, 5, "area"));
System.out.println(FunctionChaptertriangleType(5, 5, 3));
System.out.println(FunctionChaptercalculateBill(1200));
System.out.println(FunctionChaptercheckSpeed(120));
System.out.println(FunctionChaptergetGrade(85));
System.out.println(FunctionChapterlogin("admin", "1234"));

/***** OBJECTS *****/
User u = new User("Ali", 25, "Karachi");

System.out.println(u.name);
System.out.println(u.city);

// Using HashMap as JS-like object alternative
Map<String, Object> userMap = new HashMap<>();
userMap.put("name", "Ali");
userMap.put("age", 25);

System.out.println(userMap);

/***** OBJECT OPERATIONS *****/
Product p = new Product("Laptop", 50000, "Dell");

Map<String, Object> productMap = new HashMap<>();
productMap.put("name", p.name);
productMap.put("price", p.price);
productMap.put("brand", p.brand);

System.out.println(productMap);

/***** ARRAY OPERATIONS (JS map/filter equivalent) *****/
List<Product> cart = Arrays.asList(
    new Product("Laptop", 50000, "Dell"),
    new Product("Mouse", 1000, "Logitech")
);

int total = cart.stream().mapToInt(item -> item.price).sum();

List<String> names = cart.stream()
    .map(item -> item.name)
    .toList();

System.out.println("Total: " + total);
System.out.println("Names: " + names);

/***** FILTER (ACTIVE USERS) *****/
List<User> users = Arrays.asList(
    new User("Ali", 25, "Karachi"),
    new User("Sara", 22, "Lahore")
);

```

```

        List<User> filtered = users.stream()
            .filter(u1 -> u1.age > 23)
            .toList();

        System.out.println(filtered.size());
    }
}

/*****
CHAPTER 12 + 13 PORTED TO JAVA
Java Version: Uses java.util.Map, HashMap, ArrayList, Streams
Notes:
- Java does NOT support JavaScript dynamic types
- Map keys must follow type safety (Object used for mixed keys)
- No direct equivalent of Temporal API (use java.time package)
- Array behavior replaced with ArrayList / Arrays class
*****/

import java.util.*;
import java.time.*;

/* =====
   CHAPTER 12: MAP (Key-Value Collections in Java)
   ===== */

public class Main {

    public static void main(String[] args) {

        /*****
        1. CREATE MAP
        DESCRIPTION (Java):
        Map stores key-value pairs using HashMap implementation
        Keys and values must follow declared types.
        *****/
        Map<Object, Object> map = new HashMap<>();

        map.put("name", "Ali");
        map.put("age", 25);
        map.put(1, "Number Key");
        map.put(true, "Boolean Key");

        System.out.println("Initial Map: " + map);
        // OUTPUT (example):
        // {1=Number Key, name=Ali, age=25, true=Boolean Key}

        /*****
        2. put() - ADD / UPDATE VALUES
        *****/
        map.put("city", "Karachi");
        map.put("age", 30); // update

        System.out.println("After put(): " + map);
        // age updated to 30

        /*****

```



```

3. get() - FETCH VALUE
*****/
System.out.println("Name: " + map.get("name"));
System.out.println("City: " + map.get("city"));
// OUTPUT:
// Ali
// Karachi

/*****
4. containsKey() - CHECK KEY
*****/
System.out.println("Has age? " + map.containsKey("age"));
System.out.println("Has salary? " + map.containsKey("salary"));
// true / false

/*****
5. remove() - DELETE KEY
*****/
map.remove("age");
System.out.println("After remove age: " + map);

/*****
6. size - TOTAL ELEMENTS
*****/
System.out.println("Map size: " + map.size());

/*****
7. LOOP (forEach)
*****/
map.forEach((key, value) -> {
    System.out.println(key + " => " + value);
});

/*****
8. ITERATE USING ENTRY SET
*****/
for (Map.Entry<Object, Object> entry : map.entrySet()) {
    System.out.println(entry.getKey() + " : " + entry.getValue());
}

/*****
9. KEY / VALUE COLLECTIONS
*****/
System.out.println("Keys: " + map.keySet());
System.out.println("Values: " + map.values());

/*****
10. CLEAR MAP
*****/
Map<String, Integer> tempMap = new HashMap<>();
tempMap.put("a", 1);
tempMap.put("b", 2);

```

```

tempMap.clear();
System.out.println("After clear size: " + tempMap.size());
// OUTPUT: 0

/*****
11. REAL WORLD - STUDENT DATABASE
*****/
Map<Integer, String> students = new HashMap<>();
students.put(101, "Ahmed");
students.put(102, "Sara");
students.put(103, "Ali");

System.out.println("Student 102: " + students.get(102));

/*****
12. SHOPPING CART
*****/
Map<String, Integer> cart = new HashMap<>();
cart.put("Apple", 3);
cart.put("Banana", 5);

System.out.println("Apple qty: " + cart.get("Apple"));

/*****
13. OBJECT AS KEY (Java equivalent)
NOTE:
Java allows object keys but requires stable hashCode/equals
*****/
Object user = new Object();

Map<Object, String> userMap = new HashMap<>();
userMap.put(user, "User Profile Data");

System.out.println("Object Key Value: " + userMap.get(user));

/*****
CHAPTER 13: ARRAYS (Java equivalent using ArrayList)
*****/

/*****
1. ARRAY DECLARATIONS
*****/
int[] intArray = {1, 2, 3, 4, 5};
char[] charArray = {'a', 'b', 'c'};
String[] strArray = {"red", "blue", "green"};

ArrayList<Boolean> traffic = new ArrayList<>();
traffic.add(true);
traffic.add(true);
traffic.add(false);

/*****
2. LOOPING ARRAYS
*****/
int[] numArray = {3, 5, 7};

```

```

// for loop (index)
for (int i = 0; i < numArray.length; i++) {
    System.out.println(i);
}

// for-each
for (int n : numArray) {
    System.out.println(n);
}

/*****
3. MULTIPLICATION LOOP
*****/
int[] myArray = {1, 2, 3, 4};

for (int i = 0; i < myArray.length; i++) {
    int result = myArray[i] * 2;
    System.out.println("2 * value is: " + result);
}

/*****
4. ASCII GENERATION
*****/
StringBuilder sb = new StringBuilder();

for (int i = 65; i <= 122; i++) {
    sb.append((char) i);
}

System.out.println(sb.toString());

/*****
5. ARRAY CREATION METHODS
*****/
List<String> fruits = new ArrayList<>();
fruits.add("Apple");
fruits.add("Banana");
fruits.add("Mango");

System.out.println(fruits);

/*****
6. PUSH / ADD
*****/
fruits.add("Orange");

/*****
7. REMOVE LAST (POP equivalent)
*****/
fruits.remove(fruits.size() - 1);

/*****
8. UPDATE ELEMENT
*****/

```

```

*****/
fruits.set(1, "Grapes");

/*****
9. FOREACH
*****/
fruits.forEach(System.out::println);

/*****
10. MAP (TRANSFORM)
*****/
List<String> upper = new ArrayList<>();
for (String f : fruits) {
    upper.add(f.toUpperCase());
}
System.out.println(upper);

/*****
11. FILTER (Java Stream alternative)
*****/
List<String> longNames = new ArrayList<>();
for (String f : fruits) {
    if (f.length() > 5) longNames.add(f);
}
System.out.println(longNames);

/*****
12. FIND (first match)
*****/
String found = null;
for (String f : fruits) {
    if (f.startsWith("M")) {
        found = f;
        break;
    }
}
System.out.println(found);

/*****
13. INCLUDES
*****/
System.out.println(fruits.contains("Apple"));

/*****
14. INDEX OF
*****/
System.out.println(fruits.indexOf("Mango"));

/*****
15. SORT
*****/
List<Integer> numbers = Arrays.asList(40, 10, 100, 5);
Collections.sort(numbers);

```

```

System.out.println(numbers);

/*****
16. REVERSE
*****/
Collections.reverse(numbers);
System.out.println(numbers);

/*****
17. SLICE (subList)
*****/
List<String> some = fruits.subList(0, 2);
System.out.println(some);

/*****
18. JOIN
*****/
System.out.println(String.join(", ", fruits));

/*****
19. CONCAT
*****/
List<Integer> arr1 = Arrays.asList(1, 2);
List<Integer> arr2 = Arrays.asList(3, 4);

List<Integer> merged = new ArrayList<>();
merged.addAll(arr1);
merged.addAll(arr2);

System.out.println(merged);

/*****
20. REDUCE (SUM)
*****/
int sum = 0;
for (int n : numbers) {
    sum += n;
}
System.out.println(sum);

/*****
21. SHOPPING CART ANALYSIS
*****/
class Item {
    String name;
    int price;

    Item(String n, int p) {
        name = n;
        price = p;
    }
}

List<Item> cart2 = new ArrayList<>();

```

```

cart2.add(new Item("Laptop", 50000));
cart2.add(new Item("Mouse", 1000));

int total = 0;
for (Item item : cart2) {
    total += item.price;
}

System.out.println("Total: " + total);

/*****
22. IMMUTABLE ARRAY NOTE (Java difference)
Java does NOT support toReversed/toSorted/toSpliced
Alternative use copy + Collections methods.
*****/

/*****
23. MATH OPERATIONS
*****/

int[] nums = {10, 50, 20, 80, 30};

int max = Arrays.stream(nums).max().getAsInt();
int min = Arrays.stream(nums).min().getAsInt();

System.out.println("Max: " + max);
System.out.println("Min: " + min);

int totalSum = Arrays.stream(nums).sum();
double avg = totalSum / (double) nums.length;

System.out.println("Avg: " + avg);

/*****
24. OBJECT ARRAY ANALYSIS
*****/

class Product {
    String name;
    int price;

    Product(String n, int p) {
        name = n;
        price = p;
    }
}

Product[] products = {
    new Product("Laptop", 50000),
    new Product("Mouse", 1000),
    new Product("Keyboard", 3000)
};

int maxPrice = Integer.MIN_VALUE;
Product maxProduct = null;

for (Product p : products) {
    if (p.price > maxPrice) {

```

```

        maxPrice = p.price;
        maxProduct = p;
    }
}

System.out.println("Max Product: " + maxProduct.name + " " + maxProduct.price);
}
}

```

```

/*
=====

```

CHAPTER 14: SET (UNIQUE COLLECTIONS) - JAVA EDITION

TITLE: Java Set Framework (HashSet / LinkedHashSet / TreeSet)

DESCRIPTION

In Java, Set is part of java.util package and represents a collection that DOES NOT allow duplicate values.

COMMON IMPLEMENTATIONS

- HashSet !'unordered fastest
- LinkedHashSet 'maintains insertion order
- TreeSet !'sorted order

REAL-WORLD USE CASES:

- Removing duplicates from lists
- Unique user tracking
- Tag/ID/email uniqueness

```

*/

```

```

import java.util.*;

```

```

public class JavaCheatBook{

```

```

    public static void main(String[] args) {

```

```

        // =====
        // 1. CREATE A SET
        // =====
        /*
        TITLE: Creating a Set
        DESCRIPTION
        HashSet stores unique values only. Duplicates are ignored.
        */

```

```

        Set<Integer> numbersSet = new HashSet<>();
        numbersSet.addAll(Arrays.asList(1, 2, 3, 3, 4, 4, 5));

```

```

        System.out.println("Initial Set: " + numbersSet);

```

```

        // OUTPUT (order may vary):
        // Initial Set: [1, 2, 3, 4, 5]

```

```

        // =====
        // 2. ADD VALUES (add)
        // =====
        /*

```

TITLE: add() method
DESCRIPTION Inserts elements; duplicates ignored automatically
*/

```
Set<String> fruits = new HashSet<>();

fruits.add("Apple");
fruits.add("Banana");
fruits.add("Mango");
fruits.add("Apple"); // ignored

System.out.println("After add(): " + fruits);
```

```
// OUTPUT:
// After add(): [Apple, Banana, Mango]
```

```
// =====
// 3. DIFFERENT WAYS TO INITIALIZE SET
// =====
```

```
Set<String> set1 = new HashSet<>(Arrays.asList("Apple", "Banana", "Mango"));
System.out.println("From Array: " + set1);
```

```
Set<String> set2 = new HashSet<>();
set2.add("Apple");
set2.add("Banana");
set2.add("Mango");
System.out.println("Using add(): " + set2);
```

```
Set<String> set3 = new HashSet<>(Arrays.asList("A", "B", "A", "C"));
System.out.println("Duplicates Removed: " + set3);
```

```
Set<Character> set4 = new HashSet<>();
for (char c : "ABCA".toCharArray()) set4.add(c);
System.out.println("From String: " + set4);
```

```
// Java ALTERNATIVE
// Array.from() equivalent! 'Arrays.asList()' or streams
```

```
// =====
// 4. CHECK VALUE EXISTS (contains)
// =====
```

```
System.out.println("Has Banana? " + fruits.contains("Banana"));
System.out.println("Has Grapes? " + fruits.contains("Grapes"));
```

```
// OUTPUT:
// true
// false
```

```
// =====
// 5. DELETE VALUE (remove)
// =====
```

```
fruits.remove("Banana");
System.out.println("After remove: " + fruits);
```

```
// OUTPUT:
// Banana removed
```



```

// =====
// 6. SIZE OF SET
// =====
System.out.println("Set size: " + fruits.size());

// OUTPUT:
// Set size: 2

// =====
// 7. LOOP THROUGH SET
// =====
Set<String> colors = new LinkedHashSet<>(Arrays.asList("Red", "Green", "Blue"));

for (String color : colors) {
    System.out.println(color);
}

colors.forEach(c -> System.out.println("Color: " + c));

// OUTPUT:
// Red
// Green
// Blue

// =====
// 8. CONVERT SET TO ARRAY
// =====
Set<Integer> setNumbers = new HashSet<>(Arrays.asList(10, 20, 30));

Integer[] arrayNumbers = setNumbers.toArray(new Integer[0]);

System.out.println(Arrays.toString(arrayNumbers));

// OUTPUT:
// [10, 20, 30]

// =====
// 9. REMOVE DUPLICATES FROM ARRAY
// =====
/*
Java ALTERNATIVE
Use Set constructor to remove duplicates
*/

int[] arr = {1, 2, 2, 3, 4, 4, 5};

Set<Integer> unique = new LinkedHashSet<>();

for (int n : arr) unique.add(n);

System.out.println("Unique Array: " + unique);

// OUTPUT:
// [1, 2, 3, 4, 5]

```

```

// =====
// 10. CLEAR SET
// =====
Set<Integer> tempSet = new HashSet<>(Arrays.asList(1, 2, 3));

tempSet.clear();

System.out.println("After clear: " + tempSet);

// OUTPUT:
// []

// =====
// 11. SET WITH OBJECTS
// =====
Set<User> userSet = new HashSet<>();

User u1 = new User(1, "Ali");
User u2 = new User(2, "Sara");

userSet.add(u1);
userSet.add(u2);
userSet.add(u1); // duplicate reference ignored

System.out.println("User Set: " + userSet);

// NOTE:
// Java requires equals() and hashCode() override for logical duplicates

// =====
// SUMMARY (JAVA SET METHODS)
// =====
/*
add(value)      !'add element
remove(value)   !'delete element
contains()      !'check existence
size()          !'number of elements
clear()         !'remove all elements
forEach()       !'iterate elements
*/
}
}

// =====
// SUPPORT CLASS FOR SET OBJECT EXAMPLE
// =====
class User {
    int id;
    String name;

    User(int id, String name) {
        this.id = id;
        this.name = name;
    }

    @Override
    public String toString() {

```

```

        return "{id=" + id + ", name='" + name + "'}";
    }

    /*
    JAVA NOTE (IMPORTANT):
    For proper duplicate detection in Set,
    you MUST override equals() and hashCode().
    */
}

/*
=====
CHAPTER 15: JAVA DATA TYPES & BEST PRACTICES (BEGINNER GUIDE)
=====
*/

// ----- NUMBER -----
/*
Java uses primitive types instead of wrapper objects by default.
*/

int numPrim = 100;                // recommended
Integer numObj = Integer.valueOf(100); // object wrapper

System.out.println(numPrim + " -> primitive int (recommended)");
System.out.println(numObj + " -> Integer object");

// Use: calculations, counters

// ----- STRING -----
String strPrim = "Ali";
String strObj = new String("Ali");

System.out.println(strPrim + " -> String literal (recommended)");
System.out.println(strObj + " -> String object");

// Use: text processing

// ----- BOOLEAN -----
boolean flag = true;
Boolean flagObj = Boolean.TRUE;

System.out.println(flag + " -> primitive boolean (recommended)");
System.out.println(flagObj + " -> Boolean object");

// ----- ARRAY -----
int[] arr1 = {1, 2, 3};
int[][] matrix = {
    {1, 2},
    {3, 4}
};

System.out.println(Arrays.toString(arr1));
System.out.println(Arrays.deepToString(matrix));

// Use: fixed-size collections

```

```

// ----- MAP -----
Map<String, Integer> map = new HashMap<>();
map.put("id", 1);
map.put("age", 25);

System.out.println(map);

// Java NOTE:
// Map replaces JS Object/Map for key-value storage


// ----- SET -----
Set<Integer> setDemo = new HashSet<>(Arrays.asList(1, 2, 2, 3));
System.out.println(setDemo);


// ----- DATE -----
Date date = new Date();
System.out.println(date);

// Better alternative
java.time.LocalDate now = java.time.LocalDate.now();
System.out.println(now);


// ----- FUNCTION (METHOD) -----
class Demo {
    static int add(int a, int b) {
        return a + b;
    }
}

System.out.println(Demo.add(2, 3));


// ----- IMPORTANT LOOP EQUIVALENTS -----

int[] numbers = {2, 3, 5, 7, 8};

// FOR LOOP
for (int i = 0; i < numbers.length; i++) {
    System.out.println(numbers[i]);
}

// FOR-EACH LOOP (BEST)
for (int n : numbers) {
    System.out.println(n);
}

/*
JAVA VS JAVASCRIPT MAPPING:

JS for...of    !'Java for-each
JS for...in    !'Java index-based loop
JS map/filter  !'Java Streams API
JS Set         !'Java HashSet/LinkedHashSet
JS Map         !'Java HashMap
*/

```

```
/******
```

CHAPTER 16: JSON & HTTP DATA HANDLING IN JAVA

(Java Equivalent of JavaScript JSON + AJAX concepts)

NOTE:

- Java does NOT use XMLHttpRequest browser APIs.
- Instead we use:
 - ' java.net.http.HttpClient (Java 11+)
 - ' com.fasterxml.jackson or org.json for JSON handling

```
*****/
```

```
import java.net.URI;
import java.net.http.*;
import org.json.JSONObject;
```

```
public class Chapter16_JSON_HTTP{
```

```
    public static void main(String[] args) throws Exception {
```

```
        // =====
        // 1. JAVA OBJECT (Equivalent of JS Object)
        // =====
```

```
        class Person {
            String name = "Ali";
            int age = 25;
            String city = "Karachi";
        }
```

```
        Person person = new Person();
```

```
        System.out.println("JAVA OBJECT EXAMPLE: " + person.name);
        // Output (Console):
        // JAVA OBJECT EXAMPLE: Ali
```

```
        // =====
        // 2. JSON STRING
        // =====
```

```
        String jsonData = "{\"name\":\"Ali\",\"age\":25,\"city\":\"Karachi\"}";
```

```
        System.out.println("JSON STRING: " + jsonData);
        // Output:
        // JSON STRING: {"name":"Ali","age":25,"city":"Karachi"}
```

```
        // =====
        // 3. JSON !'OBJECT (Parsing)
        // =====
```

```
        JSONObject obj = new JSONObject(jsonData);
```

```
        System.out.println("PARSED NAME: " + obj.getString("name"));
        // Output:
        // PARSED NAME: Ali
```

```
        // =====
```

```

    / 4. OBJECT !'JSON (Serialization)
    // =====

    JSONObject student = new JSONObject();
    student.put("name", "Sara");
    student.put("grade", "A");

    String jsonString= student.toString();

    System.out.println("STRINGIFIED JSON: " + jsonString);
    // Output:
    // STRINGIFIED JSON: {"name":"Sara","grade":"A"}

    // =====
    // 5. TYPE CHECK (Java Equivalent)
    // =====

    System.out.println(person.getClass().getSimpleName()); // Person
    System.out.println(((Object) jsonData).getClass().getSimpleName()); // String
}
}

```

```

/*****
HTTP REQUESTS IN JAVA (AJAX Equivalent)

NOTE:
Java uses HttpClient instead of XMLHttpRequest
*****/

```

```

class HttpExamples{

    public static void main(String[] args) throws Exception {

        HttpClient client = HttpClient.newHttpClient();

        // =====
        // 1. GET REQUEST
        // =====

        HttpRequest request = HttpRequest.newBuilder()
            .uri(URI.create("https://jsonplaceholder.typicode.com/posts/1"))
            .build();

        HttpResponse<String> response =
            client.send(request, HttpResponse.BodyHandlers.ofString());

        System.out.println("GET RESPONSE");
        System.out.println(response.body());

        // =====
        // 2. GET JSON DATA + PARSE
        // =====

        HttpRequest userReq = HttpRequest.newBuilder()
            .uri(URI.create("https://jsonplaceholder.typicode.com/users/1"))
            .build();
    }
}

```

```

HttpResponse<String> userRes =
    client.send(userReq, HttpResponseBodyHandlersOfString());

JSONObject user = new JSONObject(userRes.body());

System.out.println("USER NAME: " + user.getString("name"));
System.out.println("USER EMAIL: " + user.getString("email"));
// Output:
// USER NAME: Leanne Graham
// USER EMAIL: ...

// =====
// 3. POST REQUEST
// =====

String postData = new JSONObject()
    .put("title", "My Title")
    .put("body", "My Body")
    .put("userId", 1)
    .toString();

HttpRequest postRequest = HttpRequest.newBuilder()
    .uri(URI.create("https://jsonplaceholder.typicode.com/posts"))
    .header("Content-Type", "application/json")
    .POST(HttpRequestBodyPublishersOfString(postData))
    .build();

HttpResponse<String> postResponse =
    client.send(postRequest, HttpResponseBodyHandlersOfString());

System.out.println("POST RESPONSE " + postResponse.body());

// =====
// 4. PUT REQUEST (UPDATE)
// =====

String putData = new JSONObject()
    .put("id", 1)
    .put("title", "Updated Title")
    .put("body", "Updated Body")
    .put("userId", 1)
    .toString();

HttpRequest putRequest = HttpRequest.newBuilder()
    .uri(URI.create("https://jsonplaceholder.typicode.com/posts/1"))
    .header("Content-Type", "application/json")
    .PUT(HttpRequestBodyPublishersOfString(putData))
    .build();

HttpResponse<String> putResponse =
    client.send(putRequest, HttpResponseBodyHandlersOfString());

System.out.println("PUT RESPONSE " + putResponse.body());

// =====
// 5. DELETE REQUEST
// =====

```

```

        HttpRequest deleteRequest= HttpRequest.newBuilder()
            .uri(URI.create("https://jsonplaceholder.typicode.com/posts/1"))
            .DELETE()
            .build();

        HttpResponse<String> deleteResponse=
            client.send(deleteRequest, HttpResponse.BodyHandlers.ofString());

        System.out.println("DELETE STATUS: SUCCESS (Simulated)");
    }
}

```

```

/*****
CHAPTER 17: FORM DATA HANDLING (JAVA BACKEND STYLE)

```

```

NOTE:
Java does NOT use HTML DOM directly.
Instead:
- JSP / Servlets / Spring Boot handle form input
*****/

```

```

class FormDataExample{

    static class User {
        String name;
        String email;
        String city;
        boolean active;

        User(String n, String e, String c, boolean a) {
            name = n;
            email = e;
            city = c;
            active = a;
        }
    }

    public static void main(String[] args) {

        // Simulated form input (like HTML form values)
        String name = "Ali";
        String email = "ali@gmail.com";
        String city = "Karachi";
        boolean active = true;

        // Create Java object (like JS object)
        User userData = new User(name, email, city, active);

        // Convert to JSON (for API sending)
        JSONObject jsonData = new JSONObject(userData);

        System.out.println("FORM JSON OUTPUT:");
        System.out.println(jsonData.toString());

        // Output:
        // {"name":"Ali","email":"ali@gmail.com","city":"Karachi","active":true}
    }
}

```



```
/******
```

CHAPTER 18: MODULE SYSTEM IN JAVA

```
*****
```

```
// File: MathUtils.java (module equivalent)
```

```
class MathUtils {  
  
    public static int add(int a, int b) {  
        return a + b;  
    }  
  
    public static int sub(int a, int b) {  
        return a - b;  
    }  
  
    public static int mul(int a, int b) {  
        return a * b;  
    }  
}
```

```
// File: Main.java (usage)
```

```
class MainModuleDemo {  
  
    public static void main(String[] args) {  
  
        int sum = MathUtils.add(5, 10);  
        int diff = MathUtils.sub(5, 10);  
        int product = MathUtils.mul(5, 10);  
  
        System.out.println("ADD: " + sum);  
        System.out.println("SUB: " + diff);  
        System.out.println("MUL: " + product);  
  
        // Output:  
        // ADD: 15  
        // SUB: -5  
        // MUL: 50  
    }  
}
```

```
/******
```

CHAPTER 19: TYPED ARRAYS (JAVA EQUIVALENT= PRIMITIVE ARRAYS + BYTE BUFFER)

```
*****
```

```
import java.nio.*;
```

```
class TypedArraysJava {  
  
    public static void main(String[] args) {  
  
        int[] intArray = {1, 2, 3, 4};  
        byte[] byteArray = {10, 20, 30};  
        short[] shortArray = {100, 200, 300};  
        float[] floatArray = {1.5f, 2.5f, 3.5f};  
  
        System.out.println(intArray[0]);  
        // Output: 1  
    }  
}
```

```

// LOOPING
for (int v : byteArray) {
    System.out.println(v);
}
// Output:
// 10
// 20
// 30

// MAP (TRANSFORM)
int[] doubled = new int[intArray.length];
for (int i = 0; i < intArray.length; i++) {
    doubled[i] = intArray[i] * 2;
}

System.out.println(doubled[0]); // 2

// SUM (REDUCE)
int sum = 0;
for (int v : shortArray) {
    sum += v;
}

System.out.println("SUM: " + sum);
// Output: 600
}
}

```

```

/*****
KEY DIFFERENCES SUMMARY (JAVA vs JAVASCRIPT)

' Java uses strict typing (int, String, boolean)
' No XMLHttpRequest! 'HttpClient
' No JSON global object ! 'libraries like org.json / Jackson
' No browser DOM in backend Java
' Arrays are fixed-type (int[], String[])
' Modules = classes + packages
*****/

```

```

/*****
CHAPTER 20: JAVA CORE API EQUIVALENTS OF JAVASCRIPT BUILT-IN METHODS

```

IMPORTANT NOTE:

Java does NOT provide direct equivalents for many JavaScript APIs.
Instead, Java uses:

```

' java.lang (String, Math, Object)
' java.time (modern date/time)
' java.util (List, Map, Set)
' java.util.regex (Regex)
' java.nio (ByteBuffer / Typed data)

```

This file maps JavaScript APIs to Java Standard Library equivalents

```

*****/

```

```

import java.util.*;
import java.util.regex.*;
import java.time.*;

```

```

import java.time.format.*;
import java.nio.*;
import java.text.*;

public class Chapter20_JavaCoreAPIs{

    public static void main(String[] args) {

        // =====
        // NUMBER (Java !'Integer, Double, Long wrappers)
        // =====

        int x = 10;
        double y = 10.5;

        System.out.println(Integer.isInteger(x)); // NOT DIRECT !'use type check
        System.out.println(Double.isNaN(y)); // Java equivalent

        System.out.println(Integer.parseInt("123"));
        System.out.println(Double.parseDouble("12.5"));

        System.out.println(Integer.MAX_VALUE);
        System.out.println(Integer.MIN_VALUE);

        System.out.println(Integer.toString(100));
        System.out.println(Integer.toBinaryString(10));
        System.out.println(Integer.toHexString(255));

        System.out.printf("%.2f%n", 12.3456);
        System.out.printf("%.4g%n", 1234.5678);

        // NOTE:
        // Java does NOT support:
        // num.toFixed(), toPrecision(), toExponential()
        // Alternative !'String.format / printf

        // =====
        // MATH API (java.lang.Math)
        // =====

        System.out.println(Math.abs(-10));
        System.out.println(Math.pow(2, 3));
        System.out.println(Math.sqrt(16));
        System.out.println(Math.cbrt(27));

        System.out.println(Math.round(10.6));
        System.out.println(Math.floor(10.9));
        System.out.println(Math.ceil(10.1));

        System.out.println(Math.random());

        System.out.println(Math.max(10, 20));
        System.out.println(Math.min(10, 20));

        System.out.println(Math.PI);
        System.out.println(Math.E);

        System.out.println(Math.signum(-5));
    }
}

```

```

System.out.println(Math.log(10));
System.out.println(Math.log10(10));

System.out.println(Math.sin(Math.toRadians(90)));
System.out.println(Math.cos(Math.toRadians(0)));
System.out.println(Math.tan(Math.toRadians(45)));

System.out.println(Math.hypot(3, 4)); // 5.0

// =====
// STRING (Java String class)
// =====

String str = " Hello Java ";

System.out.println(str.length());
System.out.println(str.toUpperCase());
System.out.println(str.toLowerCase());
System.out.println(str.trim());

System.out.println(str.contains("Hello"));
System.out.println(str.indexOf("H"));
System.out.println(str.lastIndexOf("l"));

System.out.println(str.replace("Java", "World"));
System.out.println(str.substring(2, 5));

System.out.println(str.startsWith(" H"));
System.out.println(str.endsWith(" "));

System.out.println(str.repeat(2));

String[] parts = str.trim().split(" ");
System.out.println(Arrays.toString(parts));

System.out.println(String.valueOf(65)); // conversion
System.out.println((char) 65); // ASCII equivalent

// NOTE:
// Java does NOT have:
// replaceAll(JS version differs), padStart/padEnd, substr
// Alternative! String.format, StringBuilder

// =====
// ARRAY (Java fixed-size arrays)
// =====

int[] arr = {1, 2, 3};

System.out.println(Arrays.toString(arr));

List<Integer> list = new ArrayList<>();
list.add(1);
list.add(2);

list.remove(Integer.valueOf(1));

```

```

System.out.println(list);

list.forEach(System.out::println);

List<Integer> mapped = list.stream()
    .map(n -> n * 2)
    .toList();

System.out.println(mapped);

Collections.sort(list);
Collections.reverse(list);

System.out.println(list);

// NOTE:
// Java does NOT have:
// map/filter/reduce directly on arrays !'use Stream API

// =====
// DATE & TIME (Modern Java API - java.time)
// =====

LocalDate date = LocalDate.now();
System.out.println(date);

LocalDateTime dt = LocalDateTime.now();
System.out.println(dt);

System.out.println(date.getDayOfMonth());
System.out.println(date.getDayOfWeek());
System.out.println(date.getMonthValue());
System.out.println(date.getYear());

System.out.println(dt.getHour());
System.out.println(dt.getMinute());

LocalDate newDate = date.plusDays(5);
System.out.println(newDate);

System.out.println(date.format(DateTimeFormatter.ISO_DATE));

// NOTE:
// Java replaces JS Date with java.time package (recommended)

// =====
// OBJECT OPERATIONS (Java Map used instead)
// =====

Map<String, Integer> obj = new HashMap<>();
obj.put("a", 1);
obj.put("b", 2);

System.out.println(obj.keySet());
System.out.println(obj.values());
System.out.println(obj.entrySet());

for (Map.Entry<String, Integer> e : obj.entrySet()) {

```

```

}

// =====
// MAP (Java HashMap)
// =====

Map<String, String> map = new HashMap<>();
map.put("id", "1");
map.put("name", "Ali");

System.out.println(map.get("name"));
System.out.println(map.containsKey("id"));
System.out.println(map.size());

for (String k : map.keySet()) {
    System.out.println(k);
}

// =====
// SET (Java HashSet)
// =====

Set<Integer> set = new HashSet<>();
set.add(1);
set.add(2);
set.add(2);

System.out.println(set);
System.out.println(set.contains(1));

set.remove(1);
System.out.println(set.size());

for (int v : set) {
    System.out.println(v);
}

// NOTE:
// Java Set = automatically unique values (same as JS Set)

// =====
// REGEX (java.util.regex)
// =====

String text = "abc123abc";

Pattern pattern = Pattern.compile("abc");
Matcher matcher = pattern.matcher(text);

while (matcher.find()) {
    System.out.println("Match at: " + matcher.start());
}

System.out.println(text.replaceAll("abc", "X"));

System.out.println(Arrays.toString(text.split("123")));

```

```

// =====
// TYPED ARRAYS (Java equivalent= NIO Buffers + primitive arrays)
// =====

    ByteBufferDemo();
}

// =====
// TYPED ARRAY DEMO (Java equivalent)
// =====
static void ByteBufferDemo() {

    ByteBuffer buffer = ByteBuffer.allocate(16);

    buffer.putInt(10);
    buffer.putInt(20);

    buffer.flip();

    System.out.println(buffer.getInt());
    System.out.println(buffer.getInt());

    // NOTE:
    // Java equivalent of TypedArray= primitive arrays + ByteBuffer
}

}

/*****
FINAL SUMMARY (IMPORTANT)

JAVASCRIPT ! 'JAVA DIFFERENCES

' Number ! 'Integer / Double / Long
' Math ! 'java.lang.Math
' String ! 'java.lang.String
' Array ! 'int[] or ArrayList<>
' Object ! 'HashMap
' Map ! 'HashMap
' Set ! 'HashSet
' Date ! 'java.time (LocalDate / LocalDateTime)
' RegExp ! 'java.util.regex.Pattern
' TypedArray ! 'ByteBuffer + primitive arrays

JAVASCRIPT FEATURES NOT DIRECTLY AVAILABLE
- toFixed() ! 'use String.format
- map/filter on arrays ! 'use Streams
- Temporal API ! 'use java.time (modern replacement)
*****/

```